SIMATS ENGINEERING

ASSESSMENT TOOL 3: Reverse Engineering Task

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1522

COURSE OUTCOME COVERED

CO4: *Analyze big data characteristics and challenges across domains including security and application aspects.* (BL4)

Dave's Taxonomy: Articulation (Level 4) **Total Marks: 30**
Question Count: 5

Code of Conduct

I _S.Blessika(192525251)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _S.Blessika

Assessment Tasks

Reverse Engineering Task

For each task, study the described big data solution, infer how it was built, identify the underlying components, and explain what design decisions were likely made.

1. A big data platform collects billions of website clicks, mobile interactions, and customer transactions every day. Reverse engineer the probable distributed storage system, ingestion framework, and batch-processing tools used. Infer how structured and semi-structured data are separated and indexed for analysis. Identify the likely stream-processing engines supporting real-time analytics and reporting. Explain the scalability and partitioning strategies probably implemented for handling massive workloads. Describe how dashboards and business intelligence systems are likely connected to the analytics layer.
2. An e-commerce platform tracks cart additions, abandoned checkouts, product views, and payment behavior across regions. Reverse engineer the likely event streaming tools, customer profiling databases, and recommendation workflow used. Infer how session data, clickstream logs, and transaction records are synchronized for real-time personalization. Identify probable caching systems, distributed query engines, and machine learning stages involved. Explain how structured order data and semi-structured browsing data are merged for analytics. Deduce the scalability and fault-tolerance decisions likely implemented. Describe how dashboards and KPI monitoring are probably generated from the pipeline.
3. A big data healthcare system processes patient histories, diagnostic images, wearable sensor streams, and prescription records. Reverse engineer the probable hybrid database architecture managing structured and unstructured healthcare information. Infer the likely data integration tools connecting hospitals, laboratories, and insurance systems. Identify the streaming and machine learning frameworks probably used for disease prediction and monitoring. Explain how privacy, encryption, and compliance requirements are likely enforced in the platform. Describe the analytics pipeline that may support clinical decision-making and population health studies.
4. A smart city platform gathers traffic sensor feeds, CCTV metadata, weather updates, and public transport activity continuously. Reverse engineer the probable edge computing setup, distributed

messaging system, and centralized storage design. Infer how geospatial analytics and streaming computations are handled for congestion prediction. Identify the likely databases used for time-series and location-aware queries. Explain how batch and real-time analytics are combined for urban planning insights. Deduce the security and access-control measures likely protecting citizen and infrastructure data. Describe how visualization systems probably deliver live operational intelligence.

5. A healthcare analytics system integrates patient records, wearable device readings, lab reports, and appointment histories. Reverse engineer the probable hybrid storage architecture handling sensitive structured and unstructured medical data. Infer the likely ETL pipelines and interoperability standards connecting hospitals and clinics. Identify how real-time monitoring and predictive health analytics are probably implemented. Explain the role of distributed processing frameworks in population-level analysis. Deduce the encryption, compliance, and audit mechanisms likely enforced for data governance. Describe how machine learning models may support diagnosis risk scoring and treatment recommendations.

Reverse Engineering Task Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Inference Accuracy	25%	Infers system components and flow with high accuracy	Mostly accurate inference	Basic inference	Weak inference
Understanding of Architecture	25%	Shows clear understanding of underlying big data architecture	Good understanding	Partial understanding	Poor understanding
Use of Technical Evidence	20%	Uses strong reasoning and technical evidence	Reasonable evidence	Limited evidence	No meaningful evidence
Depth of Analysis	15%	Analysis is deep and insightful	Reasonably deep	Basic depth	Superficial analysis
Clarity	15%	Clear and organized explanation	Generally clear	Somewhat unclear	Poorly presented

1. Question

A big data platform collects billions of website clicks, mobile interactions, and customer transactions every day. Reverse engineer the probable distributed storage system, ingestion framework, and batch-processing tools used. Infer how structured and semi-structured data are separated and indexed for analysis. Identify the likely stream-processing engines supporting real-time analytics and reporting. Explain the scalability and partitioning strategies probably implemented for handling massive workloads. Describe how dashboards and business intelligence systems are likely connected to the analytics layer.

Answer

The described platform is a large-scale **big data analytics architecture** designed to collect, store, process, and analyze billions of website clicks, mobile interactions, and customer transactions. Since the data volume is extremely high, a traditional single-server database would not be sufficient. Therefore, the platform would most probably use distributed storage, distributed processing, event-streaming systems, and a data warehouse or data lake.

1. Distributed Storage System

The platform would probably use **Hadoop Distributed File System (HDFS)** or a cloud-based data lake such as object storage.

HDFS divides large files into blocks and distributes those blocks across multiple machines. Replication is used so that if one machine fails, the data remains available on another machine.

Different types of data can be stored separately:

- Customer transaction records
- Website click logs
- Mobile application events
- Product information
- Customer behavior data
- Marketing data

Columnar formats such as **Parquet or ORC** are likely to be used for analytical data because they reduce storage requirements and improve query performance.

2. Data Ingestion Framework

The system needs to continuously receive billions of events. Therefore, **Apache Kafka** would be a probable ingestion framework.

Website and mobile applications can generate events such as:

Page View → Product Click → Search → Add to Cart → Purchase

These events are sent to Kafka topics.

For example:

- website_clicks
- mobile_events
- transactions
- customer_activity

Kafka supports high-throughput ingestion and allows multiple applications to consume the same data independently.

3. Batch Processing

For historical analysis, the platform would probably use **Apache Spark**.

Spark can process huge datasets in parallel across multiple nodes. It can perform:

- Data cleaning
- Data transformation
- Customer segmentation
- Sales analysis
- Trend analysis
- Fraud analysis
- Feature engineering
- Machine learning

Hadoop MapReduce could also be used in older architectures for large batch jobs.

4. Structured and Semi-Structured Data

The platform contains both structured and semi-structured information.

Structured data includes:

- Customer ID
- Transaction ID
- Product ID
- Transaction amount

- Date
- Payment information

This data can be stored in relational databases or data warehouse tables.

Semi-structured data includes:

- JSON clickstream events
- Mobile application logs
- Web logs
- Event metadata

This information can initially be stored in a data lake and transformed later for analytics.

5. Partitioning and Indexing

Since billions of records are involved, partitioning is essential.

The data may be partitioned by:

- Date
- Region
- Customer ID
- Product ID
- Event type

For example:

/transactions/year=2026/month=08/day=24/

This allows the analytics engine to read only the required partition instead of scanning the complete dataset.

Kafka can also divide incoming events into multiple partitions so that several consumers can process them simultaneously.

6. Real-Time Processing

Real-time analytics may be implemented using:

- Apache Flink
- Spark Streaming
- Kafka Streams

These systems can detect events almost immediately.

For example, if thousands of users suddenly search for a product, the system can identify the trend in real time.

Real-time analytics can support:

- Live sales monitoring
- Customer activity tracking
- Fraud detection
- Recommendation systems
- Website monitoring
- Alert generation

7. Scalability

The architecture would use **horizontal scaling**.

Instead of using one extremely powerful server, more servers are added to the cluster.

Scalability is achieved using:

- Kafka partitions
- Distributed storage
- Replication
- Parallel Spark processing
- Load balancing
- Cluster expansion

If data volume increases from 1 billion to 10 billion records, additional storage and processing nodes can be added.

8. Business Intelligence and Dashboards

After processing, the data can be stored in a data warehouse or analytical database.

BI tools such as Power BI, Tableau, or Metabase can connect to this analytical layer.

Dashboards may display:

- Total transactions
- Website traffic
- Daily sales
- Customer activity
- Regional sales
- Conversion rate
- Popular products
- Real-time alerts

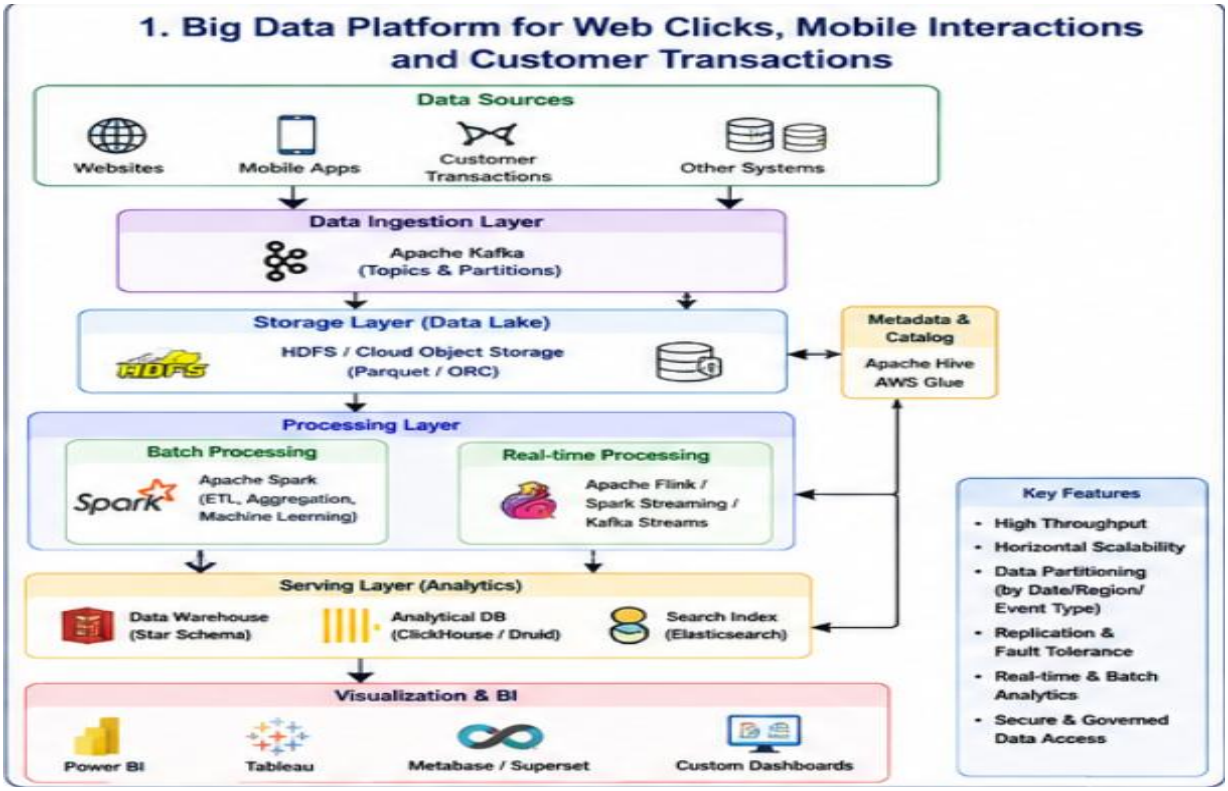
Architecture

Website + Mobile Apps + Transaction Systems
↓
Kafka / Data Ingestion
↓
HDFS / Data Lake
↓
Spark for Batch Processing + Flink for Real-Time Processing
↓
Data Warehouse / Analytical Database
↓
Power BI / Tableau / Metabase
↓
Business Dashboards and Reports

Conclusion

The most probable architecture is a combination of **Kafka, HDFS/Data Lake, Spark, Flink, Data Warehouse, and BI tools**. Distributed storage, partitioning, replication, and

parallel processing make the platform capable of handling billions of events while supporting both historical analysis and real-time business intelligence.



2. Question

An e-commerce platform tracks cart additions, abandoned checkouts, product views, and payment behavior across regions. Reverse engineer the likely event streaming tools, customer profiling databases, and recommendation workflow used. Infer how session data, clickstream logs, and transaction records are synchronized for real-time personalization. Identify probable caching systems, distributed query engines, and machine learning stages involved. Explain how structured order data and semi-structured browsing data are merged for analytics. Deduce the scalability and fault-tolerance decisions likely implemented. Describe how dashboards and KPI monitoring are probably generated from the pipeline.

Answer

The e-commerce platform is a typical **real-time big data and personalization system**. It collects customer behavior from websites and mobile applications and uses that information to understand customers and provide personalized recommendations.

The system needs to process events quickly because customer behavior changes continuously. Therefore, an event-driven architecture using Kafka, stream processing, NoSQL databases, caching, and machine learning is likely.

1. Event Streaming

Apache Kafka is a likely event-streaming platform.

The website and mobile application generate events such as:

- Product viewed
- Product searched
- Product added to cart
- Product removed from cart
- Checkout started
- Checkout abandoned
- Payment completed

These events are published to Kafka topics.

For example:

product_views

cart_events

checkout_events

payment_events

Kafka allows many consumers to use the same events for different purposes.

2. Customer Profiling Database

Customer profiles need to be accessed quickly, so a distributed NoSQL database may be used.

Possible databases include:

- MongoDB
- Cassandra

- HBase
- DynamoDB

A customer profile may contain:

Customer ID + Browsing History + Purchases + Preferences + Cart + Location

The profile can continuously change as the customer interacts with the website.

3. Session Synchronization

The system needs to connect different events belonging to the same customer session.

This can be achieved using:

- Customer ID
- Session ID
- Device ID
- Product ID
- Timestamp

For example:

Customer 102 → Viewed Laptop → Added Laptop to Cart → Abandoned Checkout

The streaming system can identify this sequence and immediately trigger a personalized action.

4. Real-Time Personalization

A real-time personalization workflow can be:

Customer Activity → Kafka → Stream Processor → Customer Profile → ML Model → Recommendation

Suppose a customer views three mobile phones but does not purchase one.

The recommendation engine can immediately suggest:

- Similar phones
- Accessories

- Discounts
- Related products

5. Caching System

Redis is a likely caching system.

It can store frequently accessed information such as:

- Customer session
- Shopping cart
- Product recommendations
- Popular products
- Product prices

Caching reduces database access and improves website response time.

6. Distributed Query Engine

For large-scale analytics, **Trino/Presto or Spark SQL** can query data stored in a data lake.

Business analysts can ask questions such as:

- Which products have the highest abandonment rate?
- Which region generates the most revenue?
- Which products are frequently viewed together?
- What percentage of customers return after receiving recommendations?

7. Machine Learning Pipeline

Machine learning can be used to create recommendation models.

The stages may include:

Data Collection → Data Cleaning → Feature Engineering → Model Training → Model Evaluation → Recommendation Generation

Possible algorithms include:

- Collaborative filtering
- Content-based filtering
- Association rules
- Matrix factorization
- Neural networks

8. Structured and Semi-Structured Data

Order information is usually structured.

Example:

Order_ID | Customer_ID | Product_ID | Amount | Date

Browsing information may be semi-structured JSON.

Example:

Customer_ID + Product_ID + Event_Type + Timestamp

An ETL/ELT pipeline can combine these datasets using common identifiers such as Customer ID and Product ID.

The result can be a unified analytical dataset.

9. Scalability

The system must handle large numbers of customers simultaneously.

Scalability can be achieved through:

- Kafka partitions
- Distributed databases
- Multiple stream-processing nodes
- Load balancing
- Horizontal scaling
- Database sharding
- Data replication

10. Fault Tolerance

Fault tolerance is important because e-commerce services operate continuously.

The system can use:

- Kafka replication
- Database replication
- Checkpointing
- Automatic recovery
- Backup storage
- Multiple processing nodes

If one server fails, another server can continue processing.

11. KPI Dashboards

Processed data can be sent to a data warehouse and connected to BI tools.

Important KPIs include:

- Total revenue
- Conversion rate
- Cart abandonment rate
- Average order value
- Number of product views
- Customer retention
- Recommendation click-through rate
- Regional sales
- Payment success rate

Architecture

Website/Mobile App
↓

Kafka Event Streaming
↓
Flink/Spark Streaming
↓
Customer Profile + Redis
↓
Machine Learning Recommendation Engine
↓
Personalized Products/Offers

At the same time:

Orders + Clickstream + Customer Data
↓
Data Lake / Warehouse
↓
Trino/Spark SQL
↓
Power BI / Tableau / Metabase

Conclusion

The probable solution is an integrated **Kafka + Flink/Spark + NoSQL + Redis + Machine Learning + Data Warehouse + BI** architecture. It provides both real-time personalization and large-scale historical analytics while maintaining scalability and fault tolerance.

3. Question

A big data healthcare system processes patient histories, diagnostic images, wearable sensor streams, and prescription records. Reverse engineer the probable hybrid database architecture managing structured and unstructured healthcare information. Infer the likely data integration tools connecting hospitals, laboratories, and insurance systems. Identify the streaming and machine learning frameworks probably used for disease prediction and monitoring. Explain how privacy, encryption, and compliance requirements are likely enforced in the platform. Describe the analytics pipeline that may support clinical decision-making and population health studies.

Answer

The healthcare system processes many different forms of medical information. These include structured patient records, unstructured medical images, semi-structured wearable data, laboratory information, and prescription records. Therefore, a **hybrid big data architecture** is likely required.

Healthcare data is also highly sensitive, so the system must provide strong security, privacy, access control, encryption, and auditing.

1. Hybrid Database Architecture

Structured healthcare information can be stored in relational databases.

Examples include:

- Patient records
- Doctor information
- Prescriptions
- Laboratory results
- Appointments
- Insurance information

These can be stored in databases such as PostgreSQL or MySQL.

Unstructured information includes:

- X-ray images
- MRI images
- CT scans
- Medical documents
- Clinical reports

These can be stored in a distributed data lake or object storage system.

2. Wearable Sensor Data

Wearable devices continuously produce:

- Heart rate

- Blood pressure
- Oxygen level
- Temperature
- Activity level
- Sleep information

Because this information is generated continuously, a streaming architecture is required.

Kafka can collect these events, while Flink or Spark Streaming can process them.

3. Healthcare Integration

Hospitals, laboratories, insurance companies, and clinics may use different systems.

Interoperability standards help connect them.

Likely standards include:

- **HL7**
- **FHIR**
- **DICOM**

FHIR can be used for exchanging patient-related healthcare information.

DICOM is commonly associated with medical imaging data.

4. ETL and Data Integration

An ETL pipeline can perform:

Extract → Transform → Validate → Integrate → Load

During transformation:

- Duplicate records are removed.
- Missing values are handled.
- Data formats are standardized.
- Patient identifiers are validated.
- Dates and timestamps are standardized.

This creates a consistent dataset for analysis.

5. Streaming Analytics

Wearable information can be processed in real time.

For example:

Wearable → IoT Gateway → Kafka → Flink → Abnormal Reading Detection → Alert

If a patient's monitored value crosses a configured clinical threshold, the system can notify authorized healthcare personnel.

6. Machine Learning

Machine learning can support:

- Disease-risk prediction
- Patient deterioration prediction
- Readmission prediction
- Medical-image analysis
- Abnormal sensor detection
- Population-health analysis

Possible models include:

- Logistic Regression
- Random Forest
- XGBoost
- Neural Networks
- CNN
- LSTM

CNN models are particularly suitable for image-based classification tasks, while time-series models can process wearable sensor data.

7. Privacy and Encryption

Healthcare data must be protected throughout its lifecycle.

The system can use:

- Encryption at rest
- Encryption in transit
- TLS-secured communication
- Strong authentication
- Role-based access control
- Data masking
- Tokenization
- Audit logging

For example, doctors may be allowed to access patient clinical information, while administrative users may have access only to billing-related information.

8. Compliance and Auditing

The platform must follow applicable healthcare privacy and data-protection requirements.

An audit system can record:

- Who accessed the data
- Which patient record was accessed
- When it was accessed
- What operation was performed

This helps identify unauthorized access.

9. Clinical Analytics

The analytics pipeline can be:

Patient Data → Data Integration → Data Lake → Data Cleaning → Feature Engineering → ML Analytics → Clinical Decision Support

Doctors can receive information such as:

- Patient risk score
- Abnormal readings
- Disease probability
- Historical trends
- Treatment outcome information

10. Population Health Studies

Historical healthcare data can be processed using Spark.

Researchers can study:

- Disease patterns
- Disease prevalence
- Regional health trends
- Treatment outcomes
- Hospital readmissions
- Patient demographics

Architecture

Hospitals + Laboratories + Wearables + Insurance Systems
↓
FHIR / HL7 / DICOM + ETL
↓
Kafka + Data Lake + Relational DB + NoSQL
↓
Spark/Flink
↓
Machine Learning
↓
Clinical Decision Support + Population Analytics
↓
Healthcare Dashboard

Conclusion

The probable healthcare architecture combines **relational databases, NoSQL databases, data lakes, Kafka, Spark/Flink, healthcare interoperability standards, and machine learning**. Security is a major design requirement, so encryption, access control, authentication, auditing, and compliance mechanisms must be integrated throughout the system.

4. Question

A smart city platform gathers traffic sensor feeds, CCTV metadata, weather updates, and public transport activity continuously. Reverse engineer the probable edge computing setup, distributed messaging system, and centralized storage design. Infer how geospatial analytics and streaming computations are handled for congestion prediction. Identify the likely databases used for time-series and location-aware queries. Explain how batch and real-time analytics are combined for urban planning insights. Deduce the security and access-control measures likely protecting citizen and infrastructure data. Describe how visualization systems probably deliver live operational intelligence.

Answer

The smart city platform continuously receives information from thousands of sensors and systems. This creates a high-volume and high-velocity data environment. Therefore, an architecture combining **edge computing, distributed messaging, streaming analytics, centralized storage, geospatial databases, machine learning, and visualization** would be appropriate.

1. Edge Computing

Traffic cameras, traffic signals, weather sensors, and transport systems can have edge devices close to the data source.

Edge computing can perform initial processing before sending information to the central platform.

For example, CCTV edge processing can identify:

- Vehicle count
- Traffic density
- Accidents

- Unusual activity

Only important metadata needs to be transmitted to the central system, reducing network traffic.

2. Distributed Messaging

Apache Kafka is a probable distributed messaging system.

Different topics can be created for:

- Traffic sensors
- CCTV metadata
- Weather
- Buses
- Trains
- Emergency events

Kafka partitions allow multiple processing nodes to consume events simultaneously.

3. Centralized Storage

A central data lake can store historical information.

Possible technologies include:

- HDFS
- Cloud object storage
- Distributed databases

The data lake can contain years of traffic, weather, transport, and infrastructure information.

4. Time-Series Database

Traffic and weather sensors produce values continuously over time.

A time-series database such as:

- InfluxDB

- TimescaleDB

can store information such as:

Timestamp + Sensor ID + Location + Vehicle Count + Average Speed

Time-series databases make historical sensor queries efficient.

5. Geospatial Database

Smart city information is strongly related to geographic locations.

PostGIS or another spatial database can support:

- Road locations
- Traffic hotspots
- Accident locations
- Bus routes
- Sensor locations
- Congestion areas

Geospatial queries can identify all traffic sensors within a particular region or road.

6. Real-Time Congestion Prediction

Real-time traffic information can be processed using:

- Apache Flink
- Spark Streaming

Machine learning models can use:

- Historical traffic
- Current vehicle count
- Vehicle speed
- Weather
- Time of day
- Day of week

- Public events

to predict future congestion.

7. Batch Analytics

Batch analytics processes historical information.

It can help city authorities understand:

- Long-term traffic patterns
- Frequently congested roads
- Transport demand
- Weather impact
- Infrastructure requirements

This information supports urban planning.

8. Combining Batch and Real-Time Analytics

The system can use both approaches.

Real-time analytics supports immediate decisions such as:

- Traffic signal changes
- Accident alerts
- Emergency response
- Public transport monitoring

Batch analytics supports long-term decisions such as:

- New road construction
- Road expansion
- Parking planning
- Public transport route planning

9. Security

Smart city systems contain sensitive infrastructure and citizen-related information.

Security can include:

- Encryption
- Authentication
- Role-based access control
- Network segmentation
- Secure APIs
- Device authentication
- Audit logging

Only authorized personnel should access sensitive CCTV and infrastructure information.

10. Visualization

Live operational dashboards can be built using:

- Grafana
- Power BI
- Tableau

Dashboards can display:

- Live traffic maps
- Traffic congestion
- Vehicle counts
- Public transport locations
- Weather
- Accident alerts
- Emergency situations

Architecture

Traffic Sensors + CCTV + Weather + Public Transport
↓

Edge Computing
↓
Kafka
↓
Flink/Spark Streaming
↓
Time-Series DB + Spatial DB + Data Lake
↓
Machine Learning
↓
Real-Time Dashboard

Historical data also flows:

Data Lake → Spark Batch Processing → Urban Planning Analytics → BI Dashboard

Conclusion

The probable smart-city architecture combines **edge computing, Kafka, Flink/Spark, time-series databases, geospatial databases, data lakes, machine learning, and BI dashboards**. This architecture provides both immediate operational intelligence and long-term urban planning insights.

5. Question

A healthcare analytics system integrates patient records, wearable device readings, lab reports, and appointment histories. Reverse engineer the probable hybrid storage architecture handling sensitive structured and unstructured medical data. Infer the likely ETL pipelines and interoperability standards connecting hospitals and clinics. Identify how real-time monitoring and predictive health analytics are probably implemented. Explain the role of distributed processing frameworks in population-level analysis. Deduce the encryption, compliance, and audit mechanisms likely enforced for data governance. Describe how machine learning models may support diagnosis risk scoring and treatment recommendations.

Answer

The described healthcare analytics system integrates data from multiple sources such as hospitals, clinics, wearable devices, laboratories, and appointment systems. Since the data is large, diverse, and sensitive, the platform would probably use a **hybrid storage**

architecture with ETL, streaming, distributed processing, machine learning, and strong security controls.

1. Hybrid Storage

Different types of healthcare data require different storage technologies.

Relational databases can store structured information such as:

- Patient ID
- Patient demographics
- Appointments
- Laboratory results
- Prescriptions
- Doctor information

A **data lake** can store:

- Medical documents
- Wearable JSON data
- Clinical reports
- Semi-structured records

Specialized object storage can be used for medical images.

2. ETL Pipeline

The ETL pipeline collects data from different healthcare systems.

The process is:

Extract → Transform → Validate → Integrate → Load

During extraction, information is collected from hospitals, clinics, laboratories, and wearable devices.

During transformation:

- Missing values are handled.

- Duplicate records are removed.
- Formats are standardized.
- Patient identifiers are matched.
- Dates and timestamps are normalized.
- Invalid records are detected.

The cleaned information is then loaded into a data warehouse or analytical data lake.

3. Interoperability

Healthcare organizations may use different software systems.

Interoperability standards such as:

- HL7
- FHIR
- DICOM

can help exchange information between systems.

FHIR can support standardized exchange of patient and clinical information, while DICOM is used for medical imaging.

4. Real-Time Monitoring

Wearable devices continuously produce health measurements.

The real-time pipeline can be:

Wearable Device → IoT Gateway → Kafka → Flink/Spark → Monitoring System

The streaming engine analyzes readings continuously.

If abnormal values are detected, an alert can be generated for authorized medical staff.

5. Predictive Health Analytics

Historical patient records and wearable readings can be used to train machine learning models.

Possible predictions include:

- Disease risk
- Patient deterioration
- Hospital readmission
- Complication risk
- Treatment outcome

Possible models include:

- Logistic Regression
- Random Forest
- XGBoost
- Neural Networks
- LSTM

6. Distributed Processing

Healthcare organizations can have millions of patient records.

Apache Spark can divide the workload across multiple processing nodes.

For example, Spark can analyze:

Millions of Patients → Disease Patterns → Risk Groups → Population Trends

Distributed processing makes large-scale analysis faster and more scalable.

7. Population-Level Analysis

Population-health researchers can use the analytical system to identify:

- Disease prevalence
- Regional disease patterns
- High-risk populations
- Treatment outcomes
- Hospital readmission trends
- Healthcare utilization

This information can support public health planning and resource allocation.

8. Encryption

Because medical data is sensitive, encryption should be used at multiple stages.

Encryption in transit: protects information while moving between systems.

Encryption at rest: protects information stored in databases and data lakes.

Strong authentication and secure communication protocols should also be implemented.

9. Access Control

Role-based access control can ensure that users only access information required for their job.

For example:

- Doctors → Relevant clinical records
- Laboratory staff → Laboratory information
- Administrative staff → Administrative information
- Analysts → Approved de-identified datasets

10. Audit Mechanisms

Every important data access should be logged.

Audit records can contain:

User → Patient/Data Accessed → Time → Action → Result

Audit trails help detect unauthorized access and support compliance requirements.

11. Machine Learning Risk Scoring

A machine learning system can combine:

- Patient history
- Age
- Laboratory results
- Previous diagnoses

- Medication history
- Appointment history
- Wearable readings

to calculate a risk score.

For example:

Patient Data → Feature Engineering → ML Model → Risk Score → Clinical Decision Support

A high-risk result can indicate that further clinical assessment may be required.

12. Treatment Recommendations

Machine learning can also analyze historical patient outcomes to provide decision-support information about possible treatments.

The system can compare:

Patient Characteristics + Previous Treatments + Outcomes

and generate recommendations for review by qualified healthcare professionals.

13. Healthcare Dashboard

The final analytics layer can be connected to dashboards.

Dashboards can show:

- Patient risk scores
- Abnormal wearable readings
- Laboratory trends
- Appointment statistics
- Disease patterns
- Population-level health indicators
- Treatment outcomes

Architecture

Hospitals + Clinics + Wearables + Laboratories
↓
ETL + HL7/FHIR + IoT APIs
↓
Kafka + Relational DB + Data Lake
↓
Spark/Flink
↓
Machine Learning Models
↓
Risk Scoring + Clinical Decision Support
↓
Healthcare Dashboard

Conclusion

The probable architecture is a combination of **hybrid databases, ETL pipelines, FHIR/HL7 interoperability, Kafka, Spark/Flink, machine learning, and BI dashboards**. Strong encryption, authentication, role-based access control, auditing, and data-governance mechanisms are essential because the platform handles highly sensitive healthcare information.